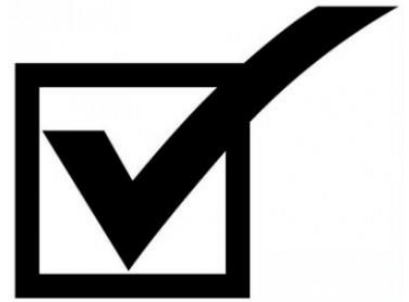


**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



Python Programming

(355)

STATE 2025

APPLICATION KNOWLEDGE:

TOTAL POINTS

_____ ***(500 points)***

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Contestant must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

Dungeon Master's Forge: Resurrecting Adventures

Elevating the dungeon crawl experience, this challenge immerses you deeper into the realm of game development. Armed with the foundational game that is *DungeonCrawler.py*, your mission now is to enhance the adventure by integrating a dynamic start menu that allows the player to either embark on a new quest or resurrect a journey from a previous save.

Contest Requirements:

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

The provided student files, *StstePython.py* and *DungeonCrawler.py*, are the files in which you will write your solution. The student file *GameSaves.txt* is the file that your solution will read from.

1. Create a folder using your *contestant number* as the name of the folder.
2. Copy the following files into this folder: *StatePython.py*, *DungeonCrawler.py*, and *GameSaves.txt*. All three files *must* be present.
3. If submitting your test with a flash drive, copy your folder to the flash drive.
4. If submitting your test online, zip your folder and upload the zipped folder to the test submission website.
5. You will need to use a local Python IDE to complete this exam. No online interpreters for Python are allowed.
6. The graders will *not* compile or alter your source code if it does not run as-is.
7. Submissions that do *not* contain source code and/or files other than the required files will *not* be graded.

What you will be doing:

1. **Include your contestant number as a comment at the top of the main source code file.**
2. Implement class ***Character*** as well as essential game functions: ***load_characters***, ***start_menu***, ***start_new_game***, and ***load_game***.
3. Make necessary adjustments to the ***start_game*** function in the *DungeonCrawler* file.
4. Initiate a start menu where the player can opt to start a new game or load a saved game.
5. Design a load screen that displays possible characters and lets the player choose one to play with.
6. Ensure the game progresses logically through three total encounters, including encounters already completed by the saved characters.

Development Standards:

1. Your Code must use a consistent variable naming convention.
2. All functions should be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a trademark of good code.

Commenting for Source Code Review (see the rubric):

1. Certain sections of your code will be graded. These gradable blocks of code can range from creating data structures, method algorithms, exception handling, and class construction.
2. The grading rubric contains a section called Source Code Review: in this section is a list of descriptions of all the graded programming concepts.
3. Each gradable item must have a comment placed at its beginning, and the comment must be prefixed with the comment flag. The flag helps the graders easily locate the code to increase the effectiveness of grading.
4. The flag will *must* use the naming convention **SC#** (NOTE: the # symbol will be replaced with sequential numbering such as **SC1, SC2, SC3**, etc.)
5. Unless stated otherwise, NO explanation in the comment with the flag is required, only the comment flag; however, any information placed in the comment could help the grader better understand and avoid any costly errors.
6. The comment flag needs to be placed near the block of code it represents.
7. **If a comment flag is not present, you will not receive credit.**
8. Here is an example: the Source Code Review has a gradable section of code for printing to the console (this is just a generic example, not on the actual rubric):
 - Rubric:
 - SC12: *print* method in the *main* class is printing the correct object _____10 pts
 - Code snippet:
#SC12 printing the car object
print(car)

Function Descriptions:

A. Class **Character** should initialize a new Character object with the following information:

- name
- weapon
- level (default value of 1)
- strength (default value of a random integer between 12 and 17)
- health (default value of a random integer between 70, 80, 90, and 100)
- gold (default value of 0)
- timestamp (default value of an empty string)

B. Function **start_menu** should display the following menu:

```
BPA Dungeon Adventure
-----
(N) New Game
(L) Load Game
Enter your choice (N/L) : █
```

The gray box is where the user should enter either an N or L (it should not be case-sensitive). The user should only be allowed to enter one of those two options. Any other value should return the phrase "Invalid choice!" and repeat the prompt.

If the user selects 'N', then the function should run the *start_new_game* function.

If the user selects 'L', then this function needs to call *load_characters* to grab the list of characters from the text file and then call *load_game* to proceed to the next phase of the game.

C. Function **load_characters** has one parameter, a filename, to read from. The function should do the following:

- Create a list that will be filled with Character objects. This list will be returned at the end of the function.
- Read each line of the file. For each line, a Character object should be made with the following information: name, weapon, level, strength, health, gold, and timestamp.
- Each Character object should be added to the list of Characters and then the completed list should be returned.

D. Function **start_new_game** should do the following, in addition to what is already provided:

- Create a new Character object with the *name* and *weapon* variables.
- Run the function *start_game* from the *DungeonCrawler* file, passing the new Character object as a parameter.

- E. Function **load_game** has a single parameter, a list of characters, and should go through the following logic:
- Display the following:

```
There are 5 available saves:
-----
1. Luna      Level 2      2024-11-27 12:00
2. Thorin    Level 1      2024-08-13 08:45
3. Mira      Level 3      2024-06-14 09:30
4. Zara      Level 2      2024-02-16 11:20
5. Eldric    Level 3      2023-03-22 20:00

Choose a save to load: █
```

The gray box is where the user should enter a number indicating which character to load. The user should only be allowed to enter an integer of 1 through 5. If the user enters any other integer, the phrase “Invalid selection. Please choose a number from the list.” should be printed and the prompt should be repeated. If the user enters a different type of value, the phrase “Invalid input. Please enter a number.” should be printed and the prompt should be repeated. You must use a while-loop as well as a try/catch to make sure a valid option is entered.

- When the user selects a valid choice, run the *display_character* function from the *DungeonCrawler* file, passing along the chosen character from the list of Character objects as a parameter. Then prompt the user for confirmation. Here is an example:

```
Name: Mira      Level: 3
Gold: 150       Weapon: Staff
Health: 80      Strength: 12

Load this character? (Y/N): █
```

The gray box is where the user should enter either Y or N (it should not be case-sensitive). The user should only be allowed to enter one of those two options. Any other value should return the phrase " Invalid input. Please enter 'Y' or 'N'." and repeat the prompt.

If the user enters ‘N’, then the function should restart, displaying the available saves and prompting the user to select an option.

If the user enters ‘Y’, run the *start_game* function from the *DungeonCrawler* file and end the loop.

- F. Function **start_game** from the file **DungeonCrawler.py** should have its for-loop altered so that the number of encounters adjusts for each character. For instance, a character at level 1 would need to go through three encounters. However, a character at level 3 would only need to get through one more encounter.

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive / uploaded as a zipped folder. _____ 20 pts
All three required files are in one folder; the folder name is your contestant ID _____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB _____ 30 pts

If the program does not execute, then the remaining items in this section receive a score of zero.

Program should accept only an 'N' or 'L' (or 'n' or 'l') for the start menu input. _____ 30 pts
When N/n is entered, welcome message should display asking for user's name. _____ 10 pts
When L/l is entered, program displays table of available saves (matching format from instructions) and prompts user to select a save to load. _____ 30 pts
Program should accept only 1, 2, 3, 4, or 5 for the input to select a save. _____ 30 pts
When valid selection is made, program displays relevant character info and prompts the user to confirm loading that character. _____ 30 pts
Program should accept only an 'Y' or 'N' (or 'y' or 'n') for the input to confirm. _____ 30 pts
When N/n is entered, program redisplayes list of saves and prompts user to make a selection. _____ 20 pts
When a Y/y is entered, program starts the game with an encounter. _____ 10 pts
Game continues to play until user gets to level 4. _____ 10 pts

Subtotal /260

Source Code Review

Contestant number is commented at the top of both Python files. _____ 10 pts

*NOTE: you **must** place the comment flag in front of the comment in your code **to get credit**. The comment flag will precede the explanation. For example, if the flag is SC1, your comment must read as "#SC1..." in front of the part of the code being reviewed. Code must work to get credit.*

SC1: Class **Character** initializes the object, with appropriate default values as given in the instructions. _____ 40 pts
SC2: Function **load_characters** takes in file name as a parameter. _____ 10 pts
SC3: Function **load_characters** reads file and creates an object out of each line _____ 30 pts
SC4: Function **load_characters** adds each object to a list and returns the list. _____ 20 pts
SC5: Function **start_menu** uses a while loop to ensure acceptable input values. _____ 10 pts
SC6: Function **start_menu** calls **load_characters** and **load_game** when L is input. _____ 20 pts
SC7: Function **start_menu** calls **start_new_game** when N is input. _____ 10 pts
SC8: Function **start_new_game** creates object and calls to **start_game** from *DungeonCrawler.py*. _____ 20 pts
SC9: Function **load_game** uses a while loop and try/catch for acceptable character selection input _____ 20 pts
SC10: Function **load_game** calls on **display_character** from *DungeonCrawler.py* and ensures acceptable input for character confirmation. _____ 20 pts
SC11: Function **load_game** calls on **start_game** from *DungeonCrawler.py*. _____ 10 pts
SC12: For-loop in function **start_game** from *DungeonCrawler.py* ensures game ends when character level reaches 4. _____ 20 pts

Subtotal /240

Total Points /500

Completed Source Code

StatePython.py

```
import random

import DungeonCrawler

class Character:

    def __init__(self, name, weapon, level=1, strength=random.randint(12, 17), health=random.randint(7, 10) * 10, gold=0, timestamp=""):

        self.name = name

        self.weapon = weapon

        self.level = level

        self.strength = strength

        self.health = health

        self.gold = gold

        self.timestamp = timestamp

def load_characters(filename):

    characters = []

    with open(filename, 'r') as file:

        for line in file:

            name, weapon, level, strength, health, gold, timestamp = line.strip().split(',')

            characters.append(Character(name, weapon, int(level), int(strength), int(health), int(gold), timestamp))

    return characters

def start_menu():

    while True:

        print("BPA Dungeon Adventure")

        print("-----")

        print("(N) New Game\n(L) Load Game")

        choice = input("Enter your choice (N/L): ").upper() # Converts the input to uppercase to handle lowercase inputs

        print()
```

```

if choice == 'L':
    characters = load_characters('GameSaves.txt')
    load_game(characters)
    break # Break out of the loop if a valid option is chosen
elif choice == 'N':
    start_new_game()
    break # Break out of the loop if a valid option is chosen
else:
    print("Invalid choice!\n")

```

```

def start_new_game():
    print("Welcome to the dungeon!\nTreasure awaits for those brave enough to venture inside...")
    name = input("Enter your character's name: ")
    weapon = input("Choose your weapon: ")
    character = Character(name, weapon)
    DungeonCrawler.start_game(character)

```

```

def load_game(characters):
    max_name_length = max(len(char.name) for char in characters)
    header = "There are " + str(len(characters)) + " available saves:"
    print()
    print(header)
    print("-" * len(header))

```

```

while True: # Loop to allow re-selection if user declines
    for i, char in enumerate(characters):
        line = f"{i+1:>5}. {char.name}\t{'Level ' + str(char.level):>5}\t{char.timestamp:>20}"
        print(line)

```

```

try:

```

```

choice = int(input("\nChoose a save to load: ")) - 1
if 0 <= choice < len(characters):
    selected_char = characters[choice]
    DungeonCrawler.display_character(selected_char) # Show chosen character details
    confirm = input("Load this character? (Y/N): ").upper()
    if confirm == 'Y':
        # Move on with that character
        DungeonCrawler.start_game(selected_char)
        break # Exit loop if confirmed
    elif confirm == 'N':
        continue # Re-display list of saves for a new choice
    else:
        print("Invalid input. Please enter 'Y' or 'N'.")
else:
    print("Invalid selection.\nPlease choose a number from the list.")
except ValueError:
    print("Invalid input. Please enter a number.")

```

Main code

start_menu()

DungeonCrawler.py

import random

def display_character(char):

print(f"\nName: {char.name}\t\tLevel: {char.level}")

print(f"Gold: {char.gold:02d}\t\tWeapon: {char.weapon}")

```
print(f"Health: {char.health}\t\tStrength: {char.strength}\n\n")
```

```
def found_loot(char):
```

```
    loot = random.choice('ggggggghhh')
```

```
    if loot == 'g':
```

```
        new_gold = random.randint(25, 150)
```

```
        char.gold += new_gold
```

```
        return f"{new_gold} gold"
```

```
    else:
```

```
        bonus = random.randint(1, 3) * 10
```

```
        char.health += bonus
```

```
        return f"a health potion. You restored {bonus} health"
```

```
def encounter(char):
```

```
    print("A monster is attacking you!")
```

```
    while True:
```

```
        try:
```

```
            attack = int(input(f"Enter:\t'1' to use your {char.weapon},\n\t'2' to run away\nChoice: "))
```

```
            if attack < 1 or attack > 2:
```

```
                print("Invalid choice!")
```

```
            else:
```

```
                break
```

```
        except ValueError:
```

```
            print("Invalid Choice!")
```

```
    if attack == 1:
```

```
        monster = random.randint(10, 20)
```

```
        result = char.strength - monster
```

```
        if result >= 0:
```

```
            reward = found_loot(char)
```

```
            input(f"\nYou defeated the monster and found {reward}!\nPress Enter to continue")
```

```
        else:
```

```

char.health -= abs(result) * 10

print(f"\nThat was rough! You lost {abs(result) * 10} health.")

if char.health <= 0:
    char.health = 0
else:
    input("Luckily you managed to get past the monster!\nPress Enter to continue")

return attack

```

```

def game_over(char):
    if char.health > 0 and char.level == 4:
        treasure = random.randint(500, 5000)
        char.gold += treasure
        print(f"\nYou made it to the treasure! You found {treasure} gold!")
    elif char.health > 0 and char.level < 4:
        print("\nYou didn't find the treasure, but you survived to fight again another day...")
    else:
        print("\nYou fought as best you could, but didn't make it.\nThe treasure waits for the next adventurer...")
    display_character(char)

```

```

def start_game(character):
    display_character(character)
    # rewrite the for loop to pick up where the character left off
    for i in range(character.level, 4):
        result = encounter(character)
        if character.health <= 0 or result == 2:
            break
        else:
            character.level += 1
            display_character(character)
    game_over(character)

```